

THE MYSTERY BUTTON — Complete Project Documentation

Project Name: The Mystery Button

Repository: [dhanush-girish/mystery-button](https://github.com/dhanush-girish/mystery-button)

Live URL: <https://mystery-button-tau.vercel.app>

Author: Dhanush Girish

Date: August 2026

Table of Contents

1. [Project Summary](#)
 2. [Technology Stack](#)
 3. [Project Architecture](#)
 4. [Directory Structure](#)
 5. [Frontend — Client](#)
 6. [Backend — API \(Serverless Functions\)](#)
 7. [Backend — Server \(Local Development\)](#)
 8. [Database Schema](#)
 9. [Authentication Flow](#)
 10. [Game Mechanics & Logic](#)
 11. [Styling & Design System](#)
 12. [Environment Variables](#)
 13. [Deployment on Vercel](#)
 14. [API Reference](#)
 15. [How It Was Built — Step by Step](#)
 16. [Dependencies — Complete List](#)
 17. [Troubleshooting & Known Issues](#)
-

1. Project Summary

The Mystery Button is an interactive web-based clicker game designed for a college event. Players log in with Google, register their name/course/batch, and then compete to click a large red button as many times as possible. A real-time leaderboard tracks the top clickers, with the **top 3 clickers winning mystery prizes**.

Key Features

-  **Google OAuth Authentication** via Clerk
-  **Clicker Game** with instant UI feedback and batched score syncing
-  **Leaderboard** with course/batch filtering
-  **Sound Effects** (with mute toggle)
-  **+1 Floating VFX** particles on each click
-  **Rank Notifications** when players enter top brackets
-  **Fully Responsive** design for mobile and desktop

- 🎮 90s Cartoon Network retro theme

2. Technology Stack

Frontend

Technology	Version	Purpose
React	19.0.0	UI library for building component-based interfaces
React Router DOM	7.1.0	Client-side routing (SPA navigation)
Vite	6.0.0	Build tool and dev server (fast HMR)
@clerk/clerk-react	5.18.0	Clerk SDK for React (Google OAuth, user sessions)
Vanilla CSS	—	Custom styling with 90s Cartoon Network theme
Google Fonts (Bangers)	—	Retro cartoon-style typography

Backend (Vercel Serverless Functions)

Technology	Version	Purpose
Vercel Serverless Functions	—	API endpoints deployed as edge functions
@clerk/backend	1.21.0	Server-side JWT token verification
@neondatabase/serverless	1.0.0	Serverless PostgreSQL client (HTTP-based)

Backend (Local Development Server)

Technology	Version	Purpose
Express.js	4.21.0	HTTP server framework for local development
@clerk/express	1.3.0	Clerk middleware for Express
cors	2.8.5	Cross-Origin Resource Sharing middleware
dotenv	16.4.7	Environment variable loading

Database

Technology	Purpose
Neon PostgreSQL	Serverless PostgreSQL database (cloud-hosted)

Deployment & Infrastructure

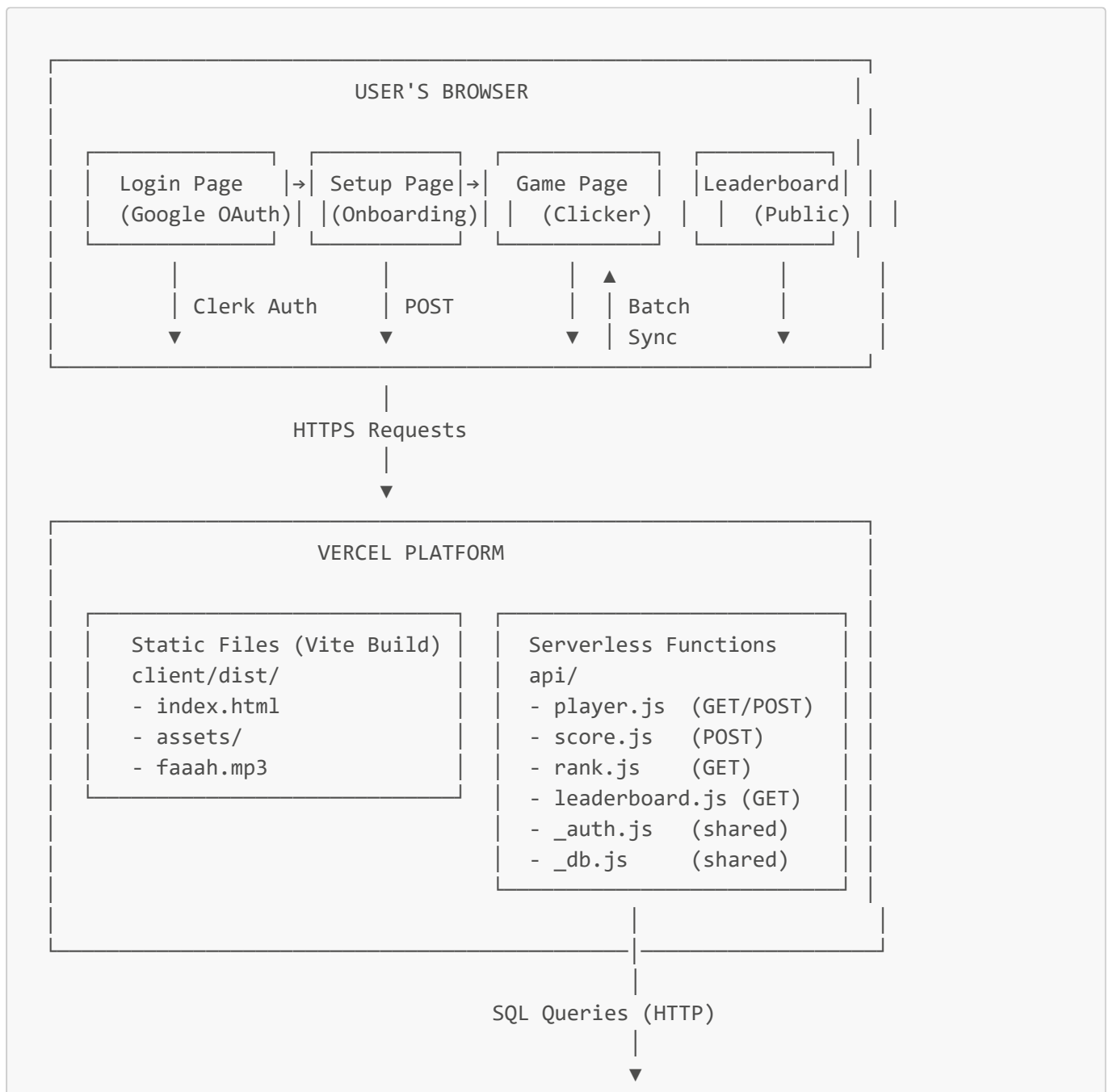
Technology	Purpose
Vercel	Hosting platform (frontend + serverless API)

Technology	Purpose
GitHub	Source code repository, triggers auto-deploy
Clerk	Authentication provider (Google OAuth)
Neon	Managed PostgreSQL database

Dev Tools

Technology	Version	Purpose
concurrently	9.1.0	Run multiple npm scripts simultaneously
@vitejs/plugin-react	4.3.0	Vite plugin for React (JSX, Fast Refresh)

3. Project Architecture



```
NEON POSTGRESQL
(Cloud Database)
```

```
Table: players
- clerk_id (PK)
- name
- course
- batch
- score
```

4. Directory Structure

```
mystery-button/
├── .env                # Environment variables (not committed)
├── .gitignore          # Git ignore rules
├── package.json         # Root package.json (monorepo scripts)
├── package-lock.json    # Root dependency lock
├── vercel.json          # Vercel deployment configuration
├── init.sql             # Database initialization script
├── api/                # ⚡ Vercel Serverless Functions
│   ├── _auth.js         # JWT token verification (shared helper)
│   ├── _db.js           # Neon database connection (shared helper)
│   ├── player.js        # GET/POST player data
│   ├── score.js         # POST batch score updates
│   ├── rank.js          # GET current player's rank
│   └── leaderboard.js   # GET leaderboard (public, filterable)
├── client/             # 🦄 React Frontend (Vite)
│   ├── index.html       # HTML entry point
│   ├── package.json     # Client dependencies
│   ├── vite.config.js   # Vite configuration
│   ├── public/
│   │   └── faaah.mp3    # Click sound effect
│   └── src/
│       ├── main.jsx      # React entry point (ClerkProvider, Router)
│       ├── App.jsx       # Route definitions
│       ├── index.css     # Complete stylesheet (919 lines)
│       ├── pages/
│       │   ├── LoginPage.jsx # Landing/login page
│       │   ├── SetupPage.jsx # Player onboarding form
│       │   ├── GamePage.jsx  # Main clicker game
│       │   └── LeaderboardPage.jsx # Public leaderboard
│       └── components/
│           ├── BigRedButton.jsx # The clickable button
│           ├── MuteToggle.jsx   # Sound on/off toggle
│           ├── PlusOneVFX.jsx   # +1 floating particles
│           └── ProtectedRoute.jsx # Auth guard for routes
```

```
├── RankToast.jsx      # Rank achievement notification
├── SearchDropdown.jsx # Searchable dropdown input
└── server/            # 🖥️ Express Server (local dev only)
    ├── index.js       # Express app with all routes
    ├── db.js          # Database connection
    ├── package.json    # Server dependencies
    └── package-lock.json # Server lock file
```

5. Frontend — Client

5.1 Entry Point — `main.jsx`

The React app is bootstrapped with three key providers:

- **React.StrictMode** — Enables development warnings
- **ClerkProvider** — Wraps the app with Clerk authentication context, using the `VITE_CLERK_PUBLISHABLE_KEY`
- **BrowserRouter** — Enables client-side routing

5.2 Routing — `App.jsx`

Route	Component	Protected?	Description
<code>/</code>	<code>LoginPage</code>	No	Landing page with Google sign-in
<code>/setup</code>	<code>SetupPage</code>	Yes	Player onboarding (name, course, batch)
<code>/game</code>	<code>GamePage</code>	Yes	Main clicker game
<code>/leaderboard</code>	<code>LeaderboardPage</code>	No	Public leaderboard with filters

Protected routes use the `ProtectedRoute` component which checks `useAuth().isSignedIn` and redirects unauthenticated users to `/`.

5.3 Pages

`LoginPage.jsx`

- Displays the game title "THE MYSTERY BUTTON" and subtitle "Do you dare to click?"
- Has a "JOIN THE BATTLE" button that opens the Clerk sign-in modal
- Auto-redirects to `/setup` if the user is already signed in

`SetupPage.jsx`

- Onboarding form collecting: **Name**, **Course** (searchable dropdown with 23 courses), and **Batch** (3 options)
- On mount, checks if the player already exists via `GET /api/player` — if yes, redirects to `/game`
- On submit, sends `POST /api/player` to register the player
- Uses the `SearchDropdown` component for course and batch selection

GamePage.jsx (Core Game Logic)

- Fetches player data on mount (`GET /api/player`)
- Displays the player's name, current score, and the Big Red Button
- **Click handling:** Each click instantly increments the UI score and spawns a +1 VFX particle
- **Batched syncing:** Clicks are accumulated in a `useRef` and synced to the server every 3 seconds via `POST /api/score`
- **Rank checking:** Every 10 seconds, fetches the player's rank via `GET /api/rank` and shows a toast notification if they enter a top bracket (Top 50 → Top 25 → Top 10)
- **Sound effects:** Plays `faaah.mp3` on each click (cloned Audio instances for overlap)
- **Page unload handling:** Flushes pending clicks using `fetch` with `keepalive: true` on `beforeunload`

LeaderboardPage.jsx

- Public page (no auth required)
- Fetches leaderboard data from `GET /api/leaderboard` with optional query filters
- Supports filtering by **Course** and **Batch** using `SearchDropdown` components
- Displays a table with Rank, Name, Course, Batch, and Score
- Top 3 players get gold/silver/bronze medal emojis and highlighted rows
- Long course names have a hover-to-scroll marquee effect

5.4 Components

Component	Props	Description
<code>BigRedButton</code>	<code>onPress(x, y)</code>	A 240px circular 3D CSS button. Uses <code>onPointerDown</code> for instant response. Passes click coordinates for VFX positioning.
<code>PlusOneVFX</code>	<code>particles: [{id, x, y}]</code>	Renders floating "+1" text at click coordinates with a CSS <code>float-up</code> animation (1s duration).
<code>MuteToggle</code>	<code>muted, onToggle</code>	Toggles sound on/off. Uses inline SVG icons for speaker/muted states.
<code>ProtectedRoute</code>	<code>children</code>	Auth guard. Checks <code>useAuth()</code> and renders children or redirects to <code>/</code> . Shows loading state while Clerk initializes.
<code>RankToast</code>	<code>message</code>	Fixed-position notification banner. Slides in from the top with a spring animation. Auto-dismissed after 4 seconds.
<code>SearchDropdown</code>	<code>options, value, onChange, placeholder</code>	Searchable dropdown with keyboard navigation (Arrow keys, Enter, Escape). Filters options as the user types. Closes on outside click.

6. Backend — API (Serverless Functions)

The `api/` directory contains Vercel Serverless Functions. Each file exports a default `handler(req, res)` function. Files prefixed with `_` are shared helpers (not exposed as endpoints).

6.1 Shared Helpers

`_db.js` — Database Connection

```
import { neon } from '@neondatabase/serverless';
const sql = neon(process.env.DATABASE_URL);
export { sql };
```

Uses the `@neondatabase/serverless` package which communicates with Neon PostgreSQL over HTTP (no persistent connections needed — perfect for serverless).

`_auth.js` — JWT Verification

```
import { verifyToken } from '@clerk/backend';
```

- Extracts the `Bearer` token from the `Authorization` header
- Verifies it using Clerk's `verifyToken()` function with the `CLERK_SECRET_KEY`
- Returns the `userId` (Clerk's `sub` claim) or sends a `401` response

6.2 API Endpoints

All authenticated endpoints include CORS headers and handle `OPTIONS` preflight requests.

Endpoint	Method	Auth	Description
<code>/api/player</code>	GET	✓	Check if the current user has a player record
<code>/api/player</code>	POST	✓	Create or update a player (upsert via <code>ON CONFLICT</code>)
<code>/api/score</code>	POST	✓	Batch-update the player's score (capped at 1000 per request)
<code>/api/rank</code>	GET	✓	Get the current player's rank (calculated via subquery)
<code>/api/leaderboard</code>	GET	✗	Get the top 100 players, with optional <code>course</code> and <code>batch</code> query filters

7. Backend — Server (Local Development)

The `server/` directory contains an **Express.js** server used only for **local development**. It mirrors the same API routes as the `api/` serverless functions but uses Express middleware (`@clerk/express`) instead of manual token verification.

Key Differences from Serverless

Feature	Serverless (<code>api/</code>)	Express (<code>server/</code>)
Auth	Manual <code>verifyToken()</code>	<code>@clerk/express</code> middleware (<code>requireAuth()</code>)

Feature	Serverless (api/)	Express (server/)
Routing	File-based (Vercel convention)	Express routes (app.get , app.post)
Database	Same (@neondatabase/serverless)	Same
Environment	process.env (Vercel injects)	dotenv (loads from .env)
CORS	Manual headers	cors() middleware

Running Locally

```
# Install all dependencies
npm run install:all

# Start both client (port 5173) and server (port 3001)
npm run dev
```

Vite's dev server proxies **/api/*** requests to **http://localhost:3001** (configured in **vite.config.js**).

8. Database Schema

PostgreSQL (Neon)

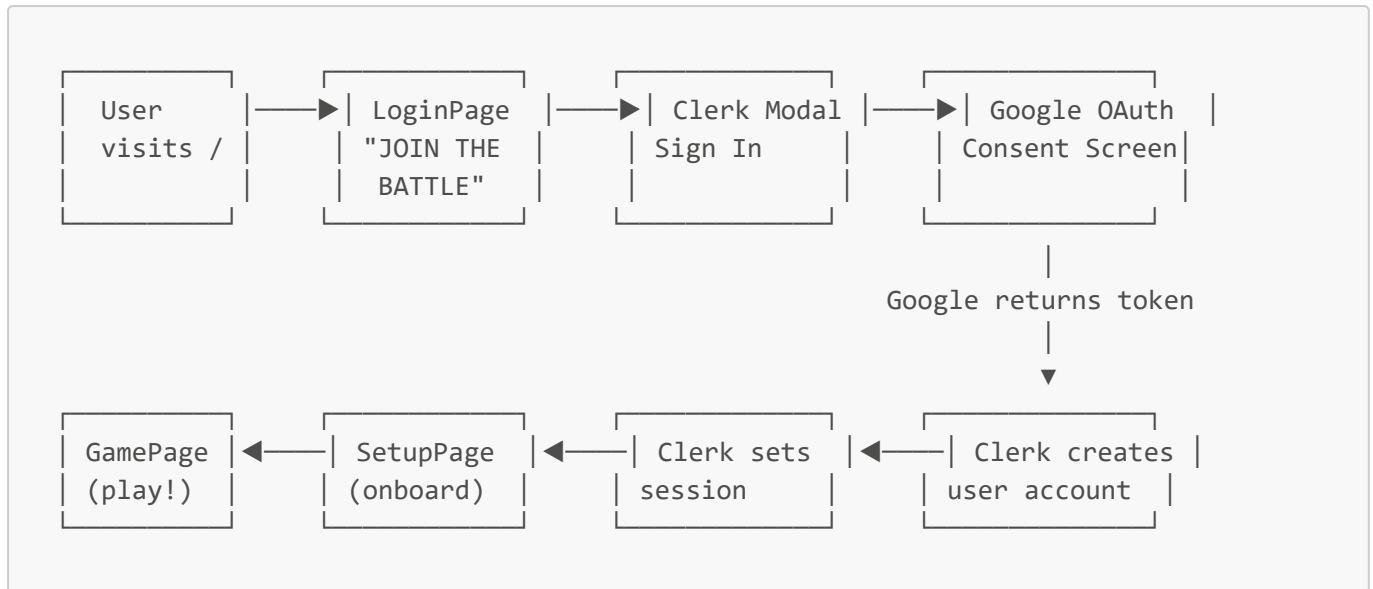
```
CREATE TABLE IF NOT EXISTS players (
  clerk_id TEXT PRIMARY KEY,      -- Clerk user ID (unique per Google account)
  name TEXT NOT NULL,             -- Player's display name
  course TEXT NOT NULL,           -- Selected course (from predefined list)
  batch TEXT NOT NULL,            -- Selected batch (e.g., "2026-2028")
  score INTEGER DEFAULT 0         -- Total click count
);

-- Performance indexes for leaderboard queries
CREATE INDEX IF NOT EXISTS idx_players_course ON players(course);
CREATE INDEX IF NOT EXISTS idx_players_batch ON players(batch);
CREATE INDEX IF NOT EXISTS idx_players_score ON players(score DESC);
```

Key Design Decisions

- **clerk_id as Primary Key:** Each Google account maps to exactly one player. The **ON CONFLICT** clause in the **INSERT** enables upsert behavior.
- **Integer Score:** Simple counter, incremented atomically in PostgreSQL (**score = score + N**).
- **Indexes on course, batch, score DESC:** Optimizes filtered leaderboard queries and sorting.

9. Authentication Flow



1. User clicks "JOIN THE BATTLE" → Opens Clerk's sign-in modal
2. User authenticates with Google OAuth
3. Clerk creates/retrieves the user and sets a session cookie + JWT
4. Client redirects to `/setup` (or `/game` if already onboarded)
5. All API calls include the JWT in `Authorization: Bearer <token>`
6. Server-side `verifyToken()` validates the JWT using `CLERK_SECRET_KEY`

10. Game Mechanics & Logic

Click Handling (Client-Side)

1. User presses the Big Red Button (via `onPointerDown` for instant feedback)
2. **Immediately:** UI score increments by 1 (optimistic update)
3. **Immediately:** `pendingClicks` ref increments by 1
4. **Immediately:** A "+1" VFX particle spawns at the click coordinates
5. **Immediately:** Sound effect plays (if not muted)

Score Syncing (Client → Server)

- Every **3 seconds**, a `setInterval` checks if `pendingClicks > 0`
- If yes, it sends `POST /api/score` with the accumulated clicks
- On failure, clicks are re-added to the pending queue (no data loss)
- On page close (`beforeunload`), remaining clicks are flushed with `keepalive: true`

Anti-Cheat Measures

- **Server-side click cap:** Maximum 1000 clicks per batch request (`Math.min(Math.floor(clicks), 1000)`)
- **Auth required:** All score updates require a valid Clerk JWT
- **Atomic updates:** Score is updated via SQL `score = score + N` (no race conditions)

Rank Notifications

- Every **10 seconds**, the client fetches the player's rank via `GET /api/rank`

- If the player crosses into a new bracket (Top 50 → Top 25 → Top 10), a toast notification appears
- The rank is calculated as a server-side subquery: `(SELECT COUNT(*) FROM players WHERE score > current_player.score) + 1`

11. Styling & Design System

Theme: 90s Cartoon Network

The entire UI is styled to evoke the nostalgic aesthetic of 90s Cartoon Network bumpers and title cards.

Design Tokens

Token	Value	Usage
Primary Pink	#ff1493	Titles, buttons, score highlights
Background Yellow	#ffd700	Body background
Polka Dot Pink	#ff69b4	Background pattern
Cyan Accent	#00cfff	Submit buttons, decorative elements
Black	#111	Borders, shadows, text
White Card	#fff	Card backgrounds

Key CSS Techniques

- **Polka Dot Background:** `radial-gradient(circle, #ff69b4 1.5px, transparent 1.5px)` with `22px` spacing
- **Comic-Style Cards:** `border: 5px solid #111` + `box-shadow: 8px 8px 0 #111` for bold offset shadows
- **3D Red Button:** Multiple `box-shadow` layers simulating depth, with `translateY` on `:active` for a press effect
- **Tilted Elements:** Subtle `transform: rotate()` on cards and titles for a hand-drawn feel
- **Decorative Pseudo-Elements:** Stars (★) and sparkles (✦) positioned around cards
- **VFX Particles:** CSS `@keyframes float-up` animation for +1 text
- **Responsive Design:** Three breakpoints (default, 768px, 400px) adjusting sizes

Typography

- **Font Family:** `Bangers` (Google Fonts)
- **Style:** Bold, uppercase, wide letter-spacing — mimics comic book lettering
- **Loading:** Preconnected via `<link rel="preconnect">` for fast font delivery

12. Environment Variables

Variable	Used By	Description
VITE_CLERK_PUBLISHABLE_KEY	Client (Vite)	Clerk publishable key — exposed to browser via <code>import.meta.env</code>
CLERK_PUBLISHABLE_KEY	Server/API	Clerk publishable key — used server-side
CLERK_SECRET_KEY	Server/API	Clerk secret key — used for JWT verification
DATABASE_URL	Server/API	Neon PostgreSQL connection string

Security Note: Only variables prefixed with `VITE_` are exposed to the browser. `CLERK_SECRET_KEY` and `DATABASE_URL` are never sent to the client.

13. Deployment on Vercel

Vercel Configuration — `vercel.json`

```
{
  "buildCommand": "cd client && npm install && npm run build",
  "outputDirectory": "client/dist",
  "framework": "vite",
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/api/$1" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

How It Works

1. **Build:** Vercel runs `cd client && npm install && npm run build` to produce the Vite static bundle in `client/dist/`
2. **Static Hosting:** Files in `client/dist/` are served as static assets
3. **Serverless Functions:** Files in `api/` are auto-detected and deployed as serverless functions
4. **Rewrites:**
 - `/api/*` requests are routed to the serverless functions
 - All other requests fall through to `index.html` (SPA client-side routing)
5. **Root Dependencies:** Vercel installs root `package.json` dependencies (`@clerk/backend`, `@neondatabase/serverless`) which the API functions import

Deployment Steps

1. Push code to GitHub (`main` branch)
2. Vercel auto-detects the push and triggers a build
3. Set environment variables in Vercel Dashboard → Settings → Environment Variables
4. Deployment is live within ~60 seconds

Post-Deployment Checklist

- ☐ Run `init.sql` in Neon SQL Editor to create the `players` table
- ☐ Add all 4 environment variables in Vercel
- ☐ Add the Vercel URL to Clerk's allowed domains (if using production mode)

14. API Reference

GET `/api/player`

Auth: Required (Bearer token)

Response (player exists):

```
{
  "exists": true,
  "player": {
    "clerk_id": "user_abc123",
    "name": "Dhanush Girish",
    "course": "MSc in Data Science and Bio AI",
    "batch": "2026-2028",
    "score": 1542
  }
}
```

Response (new user):

```
{ "exists": false }
```

POST `/api/player`

Auth: Required

Body:

```
{
  "name": "Dhanush Girish",
  "course": "MSc in Data Science and Bio AI",
  "batch": "2026-2028"
}
```

Response:

```
{ "success": true }
```

POST /api/score

Auth: Required

Body:

```
{ "clicks": 47 }
```

Response:

```
{ "success": true }
```

Clicks are capped at 1000 per request to prevent abuse.

GET /api/rank

Auth: Required

Response:

```
{  
  "rank": 12,  
  "score": 1542  
}
```

GET /api/leaderboard

Auth: Not required

Query Parameters:

- **course** (optional) — Filter by exact course name
- **batch** (optional) — Filter by exact batch

Response:

```
[  
  { "name": "Player One", "course": "MSc in CS (AI)", "batch": "2026-2028",  
    "score": 5000 },  
  { "name": "Player Two", "course": "MBA", "batch": "2025-2027", "score": 3200 }  
]
```

Returns up to 100 players, sorted by score descending.

15. How It Was Built — Step by Step

Phase 1: Project Setup

1. Initialized a monorepo with three directories: `client/`, `server/`, `api/`
2. Set up Vite with React for the client (`npx create-vite`)
3. Set up Express.js for local development server
4. Created `vercel.json` for Vercel deployment configuration
5. Set up root `package.json` with `concurrently` for running both client and server

Phase 2: Database

1. Created a Neon PostgreSQL database
2. Designed the `players` table schema with `clerk_id` as primary key
3. Added performance indexes for leaderboard queries
4. Wrote `init.sql` for reproducible setup

Phase 3: Authentication

1. Created a Clerk application with Google OAuth provider
2. Integrated `@clerk/clerk-react` on the frontend (`ClerkProvider`, `useAuth`)
3. Built the `ProtectedRoute` component for route guarding
4. Implemented server-side JWT verification using `verifyToken` from `@clerk/backend`

Phase 4: Core Pages

1. **LoginPage** — Simple landing page with Clerk sign-in modal trigger
2. **SetupPage** — Onboarding form with searchable dropdowns for course/batch selection
3. **GamePage** — Main game with the Big Red Button, score display, batched syncing, rank checking, VFX, and sound
4. **LeaderboardPage** — Public leaderboard with course/batch filtering

Phase 5: API Development

1. Built serverless functions (`api/` directory) for Vercel deployment
2. Built parallel Express routes (`server/` directory) for local development
3. Implemented score batching (client accumulates clicks, syncs every 3s)
4. Added anti-cheat measures (1000 click cap per batch)

Phase 6: Styling

1. Chose the 90s Cartoon Network theme
2. Selected "Bangers" font from Google Fonts
3. Built a comprehensive single-file CSS stylesheet (`index.css`, 919 lines)
4. Implemented responsive design with three breakpoints
5. Created 3D CSS button with depth shadows and glare effect
6. Added CSS animations for VFX particles, rank toasts, and loading states

Phase 7: Polish

1. Added sound effects (`faaah.mp3`) with mute toggle
2. Implemented rank notifications (Top 50/25/10 brackets)
3. Added `beforeunload` flush to prevent click loss on page close
4. Long course name marquee on hover in the leaderboard
5. Mobile-optimized touch handling (`touch-action: manipulation`)

Phase 8: Deployment

1. Pushed to GitHub (`dhanush-girish/mystery-button`)
2. Connected to Vercel (auto-deploy from `main` branch)
3. Set environment variables in Vercel Dashboard
4. Ran `init.sql` in Neon SQL Editor
5. Fixed `@clerk/backend` API issue (`verifyToken` is a standalone export, not a method on the clerk client)
6. Deployed successfully to `https://mystery-button-tau.vercel.app`

16. Dependencies — Complete List

Root `package.json`

Package	Version	Type	Purpose
<code>@clerk/backend</code>	<code>^1.21.0</code>	Production	Server-side Clerk SDK (JWT verification)
<code>@neondatabase/serverless</code>	<code>^1.0.0</code>	Production	Neon PostgreSQL HTTP client
<code>concurrently</code>	<code>^9.1.0</code>	Dev	Run client + server simultaneously

Client `package.json`

Package	Version	Type	Purpose
<code>@clerk/clerk-react</code>	<code>^5.18.0</code>	Production	Clerk React SDK (auth hooks, components)
<code>react</code>	<code>^19.0.0</code>	Production	UI library
<code>react-dom</code>	<code>^19.0.0</code>	Production	React DOM renderer
<code>react-router-dom</code>	<code>^7.1.0</code>	Production	Client-side routing
<code>@types/react</code>	<code>^19.0.0</code>	Dev	TypeScript types for React
<code>@types/react-dom</code>	<code>^19.0.0</code>	Dev	TypeScript types for React DOM
<code>@vitejs/plugin-react</code>	<code>^4.3.0</code>	Dev	Vite React plugin (JSX, Fast Refresh)
<code>vite</code>	<code>^6.0.0</code>	Dev	Build tool and dev server

Server `package.json`

Package	Version	Type	Purpose
@clerk/express	^1.3.0	Production	Clerk middleware for Express
@neondatabase/serverless	^1.0.0	Production	Neon PostgreSQL HTTP client
cors	^2.8.5	Production	CORS middleware
dotenv	^16.4.7	Production	.env file loading
express	^4.21.0	Production	HTTP server framework

17. Troubleshooting & Known Issues

Issue: `clerk.verifyToken` is not a function

Cause: In `@clerk/backend` v1.x, `verifyToken` is a standalone export, not a method on the Clerk client instance.

Fix: Changed `_auth.js` to use `import { verifyToken } from '@clerk/backend'` and call it directly with `{ secretKey }` options.

Issue: Application Preset stuck on "Other" in Vercel

Cause: The root directory contains a monorepo structure. Vercel can't auto-detect the framework because `vite` is only in the `client/` subdirectory.

Fix: Not a problem — `vercel.json` explicitly specifies the build command, output directory, and framework. The preset dropdown is just a UI hint.

Issue: "Something went wrong. Try again!" on Setup page

Cause: API returning non-200 status. Could be auth failure (401) or database error (500).

Fix: Check Vercel Logs tab for the actual error. Common causes: missing environment variables, uninitialized database, or Clerk domain not configured.

Issue: Clicks lost on page close

Mitigation: The `beforeunload` handler flushes pending clicks using `fetch` with `keepalive: true`. However, this is "best effort" — if the browser kills the page before the request completes, some clicks may be lost.

Issue: Clerk production mode requires custom domain

Cause: Clerk production instances require a domain you own. `*.vercel.app` is a shared domain and is rejected.

Fix: Stay in Clerk development mode for testing. For production, add a custom domain to both Vercel and Clerk.

Document generated on: August 6, 2026

Total Files in Project: 24 source files

Total Lines of Code: ~2,500+ lines